CS 4240: Compilers and Interpreters, Project BONUS, Fall 2025

Assigned: September 17, 2025, extra 5% of course grade
(5 points total)
Due on Gradescope by 11:59pm on November 24, 2025

1 Project Description

We are offering the following extra credit option for a maximum of 5 points. You will need to implement the following graph coloring register allocation optimization to reduce the number of memory load operations as an extension of Project 2. The graph coloring optimization should reduce the number of memory loads by at least 15% relative to the baseline, naive register allocator on a **single IR program of your choosing** (which should be submitted with your project).

You will need to implement a Chaitin-Briggs style graph coloring register allocator that spans an entire function/procedure. This should be enabled with a `-o` flag in your `run.sh` from project 2. You will receive the full 5 points of extra credit if you can obtain at least a 15% reduction in the number of memory loads performed by enabling your optimization option, relative to naive register allocator on a single IR program of your choosing (which you should submit with your project) and pass the hidden test case. Thus, it is possible for you to receive this extra credit even if you don't fully implement both allocators in project 2.

You will receive partial extra credit (< 5 points) depending on the best reduction you can demonstrate in the number of memory loads, relative to the total points possible. Additionally, we also have a hidden test case to validate your implementation. Thus, your final score will be based on how your allocator performs on both your submitted test case and the hidden one. (Please refer to the grading section for more details). As with Project 2, you must ensure your register allocator is correct, which will be verified by comparing the results of running the TigerIR interpreter (same as in Project 1, https://github.gatech.edu/CS-4240-Fall-2025/Project-1/tree/master/src) on the input IR program with the results of running SPIM on an output assembly program.

Global Chaitin-Briggs Style Graph Coloring Register Allocation You will need to build a register allocator that is "global" in scope, i.e., its scope spans the entire function being compiled. First, you will build the control flow graph (CFG) and perform a full CFG liveness analysis across the basic blocks. Then, you will build live ranges and an interference graph. Next, perform graph coloring using Chaitin-Briggs optimistic coloring algorithm to allocate registers. If spilling

is required, you can use any heuristic of your choice to select virtual registers to be spilled. As before, if a virtual register needs to be spilled, you should allocate space for it on the stack and insert load/store instructions for all its accesses; and, you may need to reserve some registers to temporarily hold the values for spilled registers when they are loaded from the stack.

2 Provided Code

Refer to the project 2 specification and build on your code from that submission.

3 Grading

3.1 Memory Load Reduction (4 points)

You will be graded on the % reduction in memory loads using the global graph coloring register allocator relative to the naive register allocator, on a single IR program of your choosing and the hidden test case. For the IR program that you submit, you will receive a maximum of 2 points and maximum of 2 points for the hidden test case. You will receive partial credits for both test cases separately, relative to the number of reductions in memory loads.

3.2 Correct Implementation

Your graph coloring register allocator should still be a correct implementation as the allocators in Project 2. To test this, we will have a hidden test case verifying a correct implementation, and we expect some reduction in the number of memory loads to ensure you receive the credit based on the reduction on your provided test case. If there is no change in the allocation for the hidden test case, you will receive 0 overall. Furthermore, as in Project 2, if the register allocation is incorrect and does not match the Tiger IR interpreter output, you will receive 0 for this section. Therefore, we recommend testing your implementation on other IR programs than just the one you will submit, for example, the provided test cases in Project 2, https://github.gatech.edu/CS-4240-Fall-2025/Project-2/tree/master/public_test_cases.

3.3 Organization

Your implementation should build on your Project 2 implementations. As with Project 2, your submission must include two shell scripts, `build.sh` and `run.sh`. `build.sh` should build your submission from source, as in Project 1. Make sure to document in your `report.pdf` any dependencies or version numbers required for `build.sh`. `run.sh` **must take exactly two terminal arguments—a path to an input IR file—and—a path to out.s—along with one optional flag optimization flag (`-o`). If the optimization flag is not set, the implementation should execute with whichever register you claim as your baseline. It should output an output a file `out.s` containing the generated MIPS32 code.** For example, if a Tiger-IR file is located at `path/to/file.ir`, then it should be possible to run `run.sh` as follows.

```
$ run.sh path/to/file.ir
```

```
# Produces path/to/out.s
$ run.sh path/to/file.ir -o
# Produces path/to/out.s
```

3.4 Design (1 Points)

In your final report, briefly describe the following:

1. High-level architecture of your backend, including the algorithm(s) implemented, and why you chose that approach.
2. Provide a short description of your chosen IR program. Additionally, include any usage instructions for your backend, including any dependencies or version numbers required for `build.sh`.

4 Submission

On Gradescope, submit a single ZIP file that contains:

- The complete source code of your project.
- The report.pdf file described in 3.4.
- Your chosen IR program.

5 Collaboration

Because the bonus project is an extension of Project 2, we expect the group composition to remain the same as in Project 2. If only a subset of your group plans to complete the bonus project, please notify the TAs at least *one week* before the deadline. Otherwise, all group members will receive the same score. Note that students from different groups are not permitted to collaborate on the bonus project.

Other than the above, the collaboration policy is the same as for Project 2.